CORE JAVA FOUNDATION

Course Content: Sessions 1 to 5

SESSION 1: INTRODUCTION TO JAVA & ENVIRONMENT

SESSION OVERVIEW

This session provides a comprehensive introduction to the Java programming language. It covers the historical evolution of Java, its core architecture, and the environment required to run Java applications. By the end of this session, you will understand the roles of JDK, JRE, and JVM, how Java achieves platform independence, and how to write, compile, and execute your first Java program using both the command line and an IDE.

1. Evolution of Java

Java was developed by **Sun Microsystems** (acquired by Oracle Corporation in 2010). The project began in 1991 under the guidance of **James Gosling**, Mike Sheridan, and Patrick Naughton, collectively known as the **"Green Team"**.

- **Original Purpose:** The team aimed to create a language for consumer electronic devices such as set-top boxes, televisions, and VCRs. These devices ran on various types of processors, creating a need for a language that was "Architecture Neutral" (independent of the processor).
- **Naming History:**
  - **Green Talk:** The initial name, with file extension `.gt`.
  - **Oak:** Renamed after an oak tree outside Gosling's office. This name was dropped as "Oak Technologies" already existed.
  - **Java:** Finally named "Java" in 1995, inspired by the coffee consumed by the developers. It symbolizes dynamic and lively behavior.
- **The Internet Boom:** In 1995, the World Wide Web was gaining popularity. The team realized that Java's "Write Once, Run Anywhere" capability was perfectly suited for the Web. They shifted focus from consumer electronics to internet programming, leading to Java's massive success.

2. Platform Independence: Write Once, Run Anywhere (WORA)

This is one of Java's most defining features. In traditional languages like C or C++, source code is compiled directly into machine code (binary) that is specific to the operating system (e.g., .exe for Windows). If you move that executable to Linux, it won't run.

Java solves this portability issue using **Bytecode**:

- **Source Code:** You write code in a `.java` file.
- **Compilation:** The Java Compiler (`javac`) does not create machine code. Instead, it converts source code into an intermediate format called **Bytecode** (`.class` file).
- **Execution:** This Bytecode is platform-agnostic. It can be executed on any device that has a **Java Virtual Machine (JVM)** installed. The JVM translates the

Bytecode into the specific machine code of the host computer at runtime.

3. Compiler vs Interpreter

To understand Java's execution, we must distinguish between these two translators:

Compiler	Interpreter
Scans the entire program at once and translates it into machine code or intermediate code.	Translates the program one statement at a time and executes it immediately.
Errors are reported after scanning the whole code.	Errors are reported line-by-line; execution stops at the first error.
Java's Approach: Java uses **BOTH**. The Java Compiler (`javac`) compiles source code into Bytecode. The JVM Interpreter then executes this Bytecode on the specific hardware.	

4. JDK, JRE, and JVM

These three components form the Java ecosystem:

- **JVM (Java Virtual Machine):** The engine that actually runs the application. It acts as a runtime translator, converting Bytecode into machine-specific instructions. It provides the "Sandbox" environment for security.
- **JRE (Java Runtime Environment):** This is the package required to **RUN** Java programs. It contains the JVM plus core libraries (like `java.util`, `java.lang`) and other supporting files. End-users only need the JRE.

- **JDK (Java Development Kit):** This is the full kit required to **DEVELOP** (write) Java programs. It contains the JRE plus development tools like the Compiler (`javac`), Debugger, and JavaDoc. Developers need the JDK.

5. Java Execution Flow

The lifecycle of a Java program involves three distinct stages:

1. **Creation:** The programmer writes source code in a file with the `.java` extension (e.g., `Main.java`).
2. **Compilation:** The `javac` compiler reads the `.java` file and produces a `.class` file containing Bytecode.
3. **Execution:** The JVM loads the `.class` file, verifies the bytecode for security, and executes it by translating it to the underlying OS machine code.

6. Structure of a Java Program

Every Java application must be defined inside a Class. Java is strictly Object-Oriented.

```
class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Welcome to Java Programming");
    }
}
```

7. Role of main() method

The `main` method is the entry point of any standalone Java application. The JVM looks for this specific signature to start execution.

- **public:** An access modifier. It must be public so that the JVM (which is outside the program) can access and execute it.
- **static:** This allows the JVM to call the method **without creating an object** of the class. This is critical because when the program starts, no objects exist yet.
- **void:** The return type. It means the main method does not return any value to the JVM.
- **main:** This is the specific identifier (name) the JVM looks for.
- **String args[]:** This array stores command-line arguments passed to the program during execution. It allows users to pass configuration or data to the program at startup.

8. Command-Line Compilation vs IDE Execution

You can run Java programs in two primary ways:

A. Command Line (The Fundamental Way)

This method helps you understand the underlying compilation process and classpath management.

1. Open Command Prompt or Terminal.
2. Navigate to the folder containing your file.
3. **Compile:** Type `javac HelloWorld.java`. If there are no errors, this creates a `HelloWorld.class` file.
4. **Run:** Type `java HelloWorld`. This starts the JVM, which loads the class and executes the main method.

B. IDE (The Efficient Way)

Integrated Development Environments (IDEs) like IntelliJ IDEA, Eclipse, or VS Code streamline the process.

- You simply write the code and click the "Run" button.
- The IDE handles saving, compiling, classpath configuration, and execution in the background.
- It provides features like syntax highlighting, code completion, and debugging tools.

LeetCode Practice

1. Two Sum
https://leetcode.com/problems/two-sum/

2. Add Digits
https://leetcode.com/problems/add-digits/

SESSION 2: DATA TYPES, VARIABLES & INPUT

SESSION OVERVIEW

This session explores the data storage mechanisms in Java. We will cover the different types of data (Primitive vs Reference), rules for naming variables, how to convert between types (Casting), and how to make programs interactive by taking input from the user using the Scanner class.

1. Primitive Data Types & Memory Size

Java is a **Strongly Typed** language, meaning every variable must have a declared type. Java provides 8 primitive data types to store simple values directly in memory.

Type	Size	Range	Description
byte	1 byte	-128 to 127	Smallest integer type. Useful for saving memory in large arrays.
short	2 bytes	-32,768 to 32,767	Small integers.
int	4 bytes	-2^31 to 2^31-1	The default type for whole numbers (e.g., 10, 500).

long	8 bytes	Very Large	Used for timestamps or huge numbers. Must use 'L' suffix (e.g., 9000L).
float	4 bytes	6-7 decimal digits	Single-precision decimal. Must use 'f' suffix (e.g., 10.5f).
double	8 bytes	15 decimal digits	Double-precision decimal. Default for floating-point numbers.
char	2 bytes	0 to 65,535	Stores a single Unicode character (e.g., 'A').
boolean	1 bit	true / false	Used for logical flags and conditions.

2. Reference Variables

Unlike primitives, reference variables do not store the actual data value. Instead, they store the **memory address (reference)** of an object located in the Heap memory.

- **Examples:** Arrays, Strings, User-defined Classes (like Student, Car).
- **Behavior:** If you assign one reference variable to another (e.g., `s1 = s2`), both will point to the same object in memory. Modifying one will affect the other.

3. Variable Declaration Rules & Conventions

To define a variable, you use the syntax: `DataType variableName = value;`.

- **Rules:**
  - Names are case-sensitive (`age` and `Age` are different).
  - Must start with a letter, `_` (underscore), or `$` (dollar sign). **Cannot start with a digit.**

    - No whitespace allowed within the name.
    - Cannot use Java keywords (e.g., `class`, `int`, `public`).
- **Naming Conventions (Best Practices):**
    - **Variables & Methods:** camelCase (e.g., `mySalary`, `calculateTotal()`).
    - **Classes:** PascalCase (e.g., `StudentDetails`, `Main`).
    - **Constants:** UPPER_SNAKE_CASE (e.g., `MAX_PRIORITY`).

4. Type Casting

Type casting is the process of converting a value from one data type to another.

A. Implicit Casting (Widening)

This happens automatically when converting a smaller data type to a larger data type. It is safe because there is no risk of data loss.

Order: `byte -> short -> int -> long -> float -> double`

```
int num = 100;
double d = num; // Automatically becomes 100.0
```

B. Explicit Casting (Narrowing)

This must be done manually by the programmer when converting a larger type to a smaller type. This carries the risk of **Data Loss**.

```
double pi = 3.14159;
int intPi = (int) pi; // Manually cast to int. Result is 3.
// The fractional part (0.14159) is completely lost.
```

5. Using Scanner for Real-World Inputs

To make programs interactive, we use the `Scanner` class found in the `java.util` package. It parses primitive types and strings using regular expressions.

```
import java.util.Scanner; // 1. Import the class

class UserInput {
    public static void main(String[] args) {
        // 2. Create a Scanner object connected to System.in
(Keyboard)
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your Name: ");
        String name = sc.next(); // Reads a single word

        System.out.print("Enter your Age: ");
        int age = sc.nextInt(); // Reads an integer

        System.out.print("Enter Salary: ");
        double salary = sc.nextDouble(); // Reads a decimal

        System.out.println("Details: " + name + ", " + age + ", " +
salary);
    }
}
```

LeetCode Practice

1. Reverse Integer

https://leetcode.com/problems/reverse-integer/

2. Palindrome Number

https://leetcode.com/problems/palindrome-number/

SESSION 3: OPERATORS & EXPRESSIONS

SESSION OVERVIEW

This session covers the various operators in Java that allow us to perform calculations and logic. We will discuss Arithmetic, Relational, Logical, Unary, and Ternary operators, along with operator precedence and evaluation rules.

1. Arithmetic & Relational Operators

- **Arithmetic Operators:** Used for mathematical calculations.
  - `+` (Addition), `-` (Subtraction), `*` (Multiplication).
  - `/` (Division): **Note:** Integer division truncates decimals (e.g., `10 / 3 = 3`).
  - `%` (Modulus): Returns the remainder (e.g., `10 % 3 = 1`).
- **Relational Operators:** Used to compare two values. They always return a `boolean` (true/false).
  - `==` (Equal to), `!=` (Not Equal to).
  - `>` (Greater than), `<` (Less than), `>=`, `<=`.

2. Logical Operators and Short-circuiting

These are used to combine multiple boolean conditions.

- `&&` (Logical AND): Returns true only if **both** conditions are true.
- `||` (Logical OR): Returns true if **at least one** condition is true.
- `!` (Logical NOT): Reverses the boolean value (true becomes false, false becomes true).

Short-circuiting: Java optimizes logical operations.
- In `cond1 && cond2`, if `cond1` is false, Java stops and does not check `cond2` because the result is already known (false).
- In `cond1 || cond2`, if `cond1` is true, Java stops and does not check `cond2`.

3. Unary Operators (++, --)

These operators act on a single operand to increment or decrement its value by 1.

- **Pre-increment (++a):** Increments the value first, THEN uses it in the expression.
- **Post-increment (a++):** Uses the current value in the expression first, THEN increments it.

```
int a = 10;
System.out.println(a++); // Prints 10, then 'a' becomes 11
System.out.println(++a); // 'a' becomes 12, then Prints 12

// Complex Expression Example
int x = 5;
// Logic: x++ uses 5 (x becomes 6). ++x becomes 7 (uses 7).
// Result: 5 + 7 = 12
int result = x++ + ++x;
System.out.println(result); // 12
```

4. Ternary Operator

The Ternary operator is a shorthand for writing simple `if-else` statements. It is the only operator in Java that takes three operands.

Syntax: `variable = (condition) ? value_if_true : value_if_false;`

```
int a = 10, b = 20;
// If a > b, assign a to max, else assign b
int max = (a > b) ? a : b;
System.out.println("Max is: " + max);
```

5. Operator Precedence & Evaluation Order

When an expression has multiple operators, Precedence rules decide which operator is executed first.

- Multiplicative operators (`*`, `/`, `%`) have higher precedence than Additive operators (`+`, `-`).
- Example: `10 + 5 * 2` equals `20`, not `30`, because multiplication happens first.
- Parentheses `()` can be used to override precedence.

LeetCode Practice

1. Single Number

https://leetcode.com/problems/single-number/

2. Plus One

https://leetcode.com/problems/plus-one/

SESSION 4: CONDITIONAL STATEMENTS

SESSION OVERVIEW

This session focuses on decision-making logic in Java. We cover standard conditional structures like if, else-if, and nested if, as well as the Switch statement for multi-way branching. We also discuss common pitfalls like fall-through behavior.

1. If, If-Else, and Nested If

These statements control the flow of execution based on boolean conditions.

A. Else-If Ladder

Used when you have multiple conditions to check sequentially.

```
int marks = 85;

if (marks >= 90)
    System.out.println("A");
else if (marks >= 75)
    System.out.println("B");
else
    System.out.println("C");
```

B. Nested If

An `if` statement inside another `if` statement. Used for complex validation.

```
int age = 25;
int weight = 55;

if (age > 18) {
    if (weight > 50) {
        System.out.println("Eligible to donate blood");
    } else {
        System.out.println("Underweight");
    }
}
```

2. Switch-Case Statement

The `switch` statement is a cleaner alternative to long `else-if` chains when testing a single variable against multiple constant values.

- Supported types: `byte`, `short`, `char`, `int`, `String` (Java 7+), and `Enum`.

Example: Menu-Driven Logic

Menu-driven programs allow users to select an operation from a list.

```
int choice = 2;
switch(choice) {
    case 1:
        System.out.println("Selected: Start Game");
        break;
    case 2:
        System.out.println("Selected: Load Game");
        break;
    case 3:
        System.out.println("Selected: Exit");
        break;
    default:
        System.out.println("Invalid Selection");
}
```

3. Avoiding Fall-Through Errors

If you omit the `break` keyword, Java execution "falls through" to the next case, executing all subsequent cases until a break is found or the switch ends. This is a common bug but can sometimes be used intentionally (e.g., to group cases).

```java
// Example of Fall-through logic (Checking Weekend)
int day = 6;
switch(day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        System.out.println("Weekday");
        break;
    case 6:
    case 7:
        System.out.println("Weekend"); // Executed for 6 or 7
        break;
}
```

LeetCode Practice

1. Power of Two
https://leetcode.com/problems/power-of-two/

2. Number of Steps to Reduce a Number to Zero
https://leetcode.com/problems/number-of-steps-to-reduce-a-number-to-zero/

SESSION 5: LOOPS (ITERATION)

SESSION OVERVIEW

This session is about repetition logic. We explore the three main types of loops (for, while, do-while), how to control them using break and continue, and how to use nested loops for complex tasks like pattern printing and mathematical series.

1. The Three Loop Types

A. For Loop

Best used when the number of iterations is fixed and known beforehand.

```
// Print numbers 1 to 5
for(int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

B. While Loop

Best used when the number of iterations is unknown and depends on a condition. The condition is checked **before** the loop body executes.

```
int i = 1;
while(i <= 5) {
    System.out.println(i);
    i++;
}
```

C. Do-While Loop

Similar to `while`, but the condition is checked **after** the loop body. This guarantees that the loop runs **at least once**, even if the condition is false initially.

```
int i = 1;
do {
    System.out.println("This executes at least once");
    i++;
} while(i < 1);
```

2. Loop Control: Break & Continue

Break Statement

Break: Exits the loop immediately. Commonly used to stop searching when an item is found.

```
// Finding 50 in a loop
for(int i=10; i<=100; i+=10) {
    if(i == 50) {
        System.out.println("Found 50, exiting loop.");
        break; // Stops execution
    }
    System.out.println("Checking " + i);
}
```

Continue Statement

Continue: Skips the current iteration and jumps to the next one. Useful for skipping invalid data.

```
// Skipping odd numbers
for(int i=1; i<=5; i++) {
    if(i % 2 != 0) {
        continue; // Skip rest of the code for odd numbers
    }
    System.out.println("Even number: " + i);
}
```

3. Nested Loops & Patterns

A loop inside another loop is called a nested loop. This is essential for working with 2D arrays (matrices) and generating visual patterns.

Example: Right Angle Triangle Pattern

```
/*
*
* *
* * *
*/
for(int i=1; i<=3; i++) {       // Outer loop (Rows)
    for(int j=1; j<=i; j++) {   // Inner loop (Columns)
        System.out.print("* ");
    }
    System.out.println();       // Move to new line
}
```

4. Mathematical Series

Loops are heavily used for mathematical problems like calculating Factorials, Fibonacci series, or Prime numbers.

```
// Example: Calculating Factorial of 5 (5! = 120)
int n = 5;
int fact = 1;
for(int i=1; i<=n; i++) {
    fact = fact * i;
}
System.out.println("Factorial: " + fact);
```

LeetCode Practice

1. Fizz Buzz

https://leetcode.com/problems/fizz-buzz/

2. Count Primes

https://leetcode.com/problems/count-primes/